

Laporan Tugas Besar
IF25-21013: Strategi Algoritma

Penerapan Algoritma Greedy

Dosen Pengampu:
Imam Eko Wicaksono, S.Si, M.Si.



Dibuat Oleh:
Albert Christian Sihalocho (124140178)
Jalaludin Mufadhol Al Faruq (124140154)
Jundi Lamtara (124140190)

TEKNIK INFORMATIKA
INSTITUT TEKNOLOGI SUMATERA
2026

DAFTAR ISI

BAB 1 Deskripsi Tugas	2
BAB 2 Landasan Teori.....	3
2.1 Algoritma Greedy	3
2.2 Implementasi Algoritma Greedy	3
BAB 3 Aplikasi Strategi Greedy	4
3.1 Penerapan Dasar	4
3.2 Alternatif Greedy pada Robocode.....	4
3.3 Strategi Utama Kelompok.....	5
BAB 4 Implementasi dan Pengujian	6
4.1 Implementasi Strategi.....	6
BAB 5 Kesimpulan dan Saran	14
5.1. Kesimpulan.....	14
5.2. Saran	15
Lampiran	16
Daftar Pustaka	17

BAB 1

Deskripsi Tugas

Robocode adalah permainan pemrograman yang bertujuan untuk membuat kode bot dalam bentuk tank virtual untuk berkompetisi melawan bot lain di arena. Pertempuran Robocode berlangsung hingga bot-bot bertarung hanya tersisa satu seperti permainan Battle Royale, karena itulah permainan ini dinamakan Tank Royale.

Dalam permainan ini, pemain berperan sebagai programmer bot dan tidak memiliki kendali langsung atas permainan. Pemain hanya bertugas untuk membuat program yang menentukan logika pada bot. Program yang dibuat akan berisi instruksi tentang cara bot bergerak, mendeteksi bot lawan, menembakkan senjatanya, serta bagaimana bot bereaksi terhadap berbagai kejadian selama pertempuran.

Pada tugas besar pertama ini, mahasiswa diminta untuk membuat sebuah bot yang nantinya akan dipertandingkan satu sama lain. Algoritma yang wajib digunakan untuk mengatur kemampuan bot adalah algoritma greedy, di mana strategi pengambilan keputusan harus berdasarkan pemilihan aksi terbaik secara lokal pada setiap langkah. Mahasiswa diminta untuk tidak hanya mengimplementasikan bot tersebut, tetapi juga menjelaskan strategi greedy yang digunakan dalam laporan ini secara rinci, mulai dari proses pemetaan persoalan ke elemen-elemen greedy hingga analisis efektivitas strategi tersebut.

Tujuan akhir dari tugas besar ini adalah untuk menghasilkan bot dengan strategi greedy terbaik yang mampu bersaing dengan bot dari kelompok lain dalam kompetisi. Selain implementasi teknis, laporan ini juga mencakup penjelasan teoritis, eksplorasi strategi alternatif, analisis efektivitas, serta pengujian terhadap solusi yang dibuat. Oleh karena itu, pemahaman mendalam tentang algoritma greedy dan kemampuan mengimplementasikannya ke dalam konteks permainan menjadi aspek utama yang diuji dalam tugas besar ini.

BAB 2

Landasan Teori

2.1 Algoritma Greedy

Secara fundamental, algoritma greedy adalah algoritma yang memulai dari posisi awal sebuah masalah hingga penyelesaiannya. Tujuan akhir yang ingin dicapai dengan Algoritma Greedy adalah untuk mendapatkan Solusi terbaik secepat mungkin. Ketika tujuan sudah tercapai atau tidak ada Langkah yang dapat diambil, maka algoritma ini berhenti. Tentu saja, Algoritma Greedy tidaklah sempurna dan mempunyai kelemahannya tersendiri. Pada algoritma ini, solusi akhir yang didapat tidak selalu menjadi solusi yang terbaik. Algoritma ini juga tidak bisa mencari nilai solusi maksimum atau minimum, hanya solusi yang dapat dipenuhi pada range tertentu. Hal ini dikarenakan greedy memprioritaskan menyelesaikan semuanya dengan cara cepat dan mengharapkan hasil terbaik.

Algoritma ini berjalan dengan solusi inisial dari masalah yang ada; mencari elemen solusi dari seluruh solusi yang ada dan maju menuju tujuan yang dicari dengan menggabungkan semua elemen solusi menjadi satu solusi yang rangkum dari masalah yang ada. Hal ini berarti algoritma greedy akan mencari semua kemungkinan solusi yang ada dan mengambil solusi yang menurutnya terbaik. Seperti yang diterangkan sebelumnya juga, solusi terbaik dari Algoritma Greedy tidak selalu menjadi solusi yang paling efisien. (Chen, 2022)

2.2 Implementasi Algoritma Greedy

Strategi greedy yang diterapkan pada percobaan ini mengutamakan ketepatan pengambilan keputusan berdasarkan kondisi arena terkini yang diperoleh melalui proses scanning, tanpa melakukan pencarian menyeluruh terhadap seluruh kemungkinan state di masa depan. Meski demikian, bot pada percobaan tetap memanfaatkan prediksi sederhana untuk memperkirakan posisi musuh pada saat penembakan dilakukan. Pendekatan ini terbukti efektif dalam lingkungan permainan yang bersifat dinamis dan terus berubah. Akan tetapi, algoritma greedy tidak selalu menghasilkan solusi yang optimal. Sebagai contoh, bot berpotensi mengalami kondisi stuck ketika menabrak tembok akibat tidak adanya mekanisme penanganan khusus, atau terlalu berfokus pada satu lawan tertentu sehingga mengabaikan ancaman dari bot lain yang memberikan damage lebih besar.

BAB 3

Aplikasi Strategi Greedy

3.1 Penerapan Dasar

Di dalam Robocode, terdapat beberapa nilai dasar yang dapat digunakan sebagai acuan untuk menerapkan Algoritma Greedy didalamnya. Robocode menggunakan sistem 'Point' untuk memperlihatkan seberapa efektif sebuah bot dapat bekerja. Semakin tinggi point tersebut, maka semakin besar kemungkinan bot terkait untuk mencapai kemenangan. Maka pada aplikasinya, bot akan disesuaikan untuk mendapatkan point terbanyak pada skirmish yang akan dilakukan.

Dalam penerapannya, bot dapat melakukan atau mengikuti beberapa aksi untuk menghasilkan hasil terbaik. Aksi-aksi ini meliputi:

- Aksi yang diambil oleh bot akan mempengaruhi point akhir yang dapat diperoleh oleh bot. Keputusan untuk maju, mundur, atau mengambil posisi yang strategis sangatlah mempengaruhi hasil akhir dari percobaan.
- Kondisi dari *battlefield* juga akan mempengaruhi hasil akhir dari percobaan. Satu perubahan variabel pada field akan mempengaruhi semua alur jalannya pertarungan. Karena Algoritma Greedy mengejar hasil terbaik dengan cepat, kondisi buruk dapat membuat algoritma ini menjadi lebih tidak optimal dari biasanya karena variabel acak yang ada.

3.2 Alternatif Greedy pada Robocode

Strategi Greedy dapat diterapkan dengan banyak cara dan variabel aksi yang bisa didapatkan atau dilakukan dalam waktu sangatlah bervariasi. Dengan *heuristic* yang berbeda, akan didapatkan pula hasil yang berbeda. Setiap *heuristic* yang berbeda akan membuat setiap aksi dari bot yang berbeda juga akan unik pada botnya masing-masing. Pada kelompok kami, terdapat empat *heuristic* yang diimplementasikan pada empat bot berbeda.

Dimulai dari bot *Kamikaze*. Bot ini menggunakan pendekatan greedy dimana bot akan selalu mencoba untuk melakukan *ramming* terhadap bot lainnya yang ada di *battlefield*. Bot ini akan mengunci satu target hingga target tersebut hancur, lalu akan mencari target selanjutnya didalam area *scanning*-nya. Secara simpel, *heuristic* dari bot ini hanyalah menjadi agresif, menerapkan Algoritma Greedy dengan mencoba mendapatkan point lebih banyak dari *damage ramming* yang dilakukan ke bot lawan. Bot ini dapat menghasilkan point yang substansial, namun lemah ketika bot lawan dapat menghindar dengan baik.

Selanjutnya terdapat bot *RotasiBumi*. Bot ini menggunakan pendekatan greedy dimana bot akan mencoba untuk adaptasi dengan kondisi *battlefield*. Bot akan selalu berputar dengan *track* berbentuk lingkaran untuk menghindari *bullet* dari bot lawan. Sesuai dengan jarak bot ini dengan targetnya, *bullet size* akan disesuaikan. *Heuristic* yang digunakan pada bot ini adalah bermain aman dengan melakukan *dodge* dan mencoba untuk mendapatkan point sebanyak mungkin dengan adaptasi *attack*-nya. Bot ini sangatlah kuat dalam posisi 1 vs 1, namun kekuatannya jatuh ketika harus melawan lebih dari 1 bot.

Lalu, terdapat bot *Sweepy*. Bot ini menggunakan pendekatan greedy dimana bot akan mencoba untuk menghindari pertempuran jarak dekat sebaik mungkin dan melakukan lock pada satu target. Bot ini akan bergerak memutar arena, menghindari kontak langsung yang akan terjadi di tengah arena. Ketika bot di-*ram* atau melakukan *ram* pada bot lain, maka Bot ini akan rescanned dan fokus pada target terdekat yang dideteksi. *Heuristic* yang digunakan pada bot ini adalah bermain aman seperti bot sebelumnya, namun akan menjadi agresif ketika melakukan kontak langsung dengan bot lawan. Bot ini memiliki *survival rate* yang tinggi, namun umumnya tidak mendapatkan skor tinggi karena jarang melakukan kontak langsung.

Terakhir terdapat bot *RyoikiTenkai*. Bot ini menggunakan pendekatan Greedy Anti-Gravity Movement, yaitu bergerak menuju bagian arena dengan *engagement* paling sedikit sambil menghindari proyektil lawan. Ketika kondisi sudah berubah menjadi 1v1, bot akan menggunakan pendekatan Stop-and-Go, yaitu bergerak zig-zag dan mengubah arah secara tiba-tiba saat musuh menembak agar sulit diprediksi dan terkena peluru. *Heuristic* pada bot ini adalah untuk bermain aman seperti pada bot-bot sebelumnya, namun berbeda dengan bot lainnya, mekanisme bot ini sudah berkerja dengan lebih baik. Bot ini memiliki *survival rate* yang tinggi dan umumnya selalu menjadi juara pada percobaan yang dilakukan

3.3 Strategi Utama Kelompok

Berdasarkan hasil analisis, perancangan, dan simulasi beberapa purwarupa bot yang telah dikembangkan (Kamikaze, RotasiBumi, dan Sweepy), kelompok kami menetapkan bot *RyoikiTenkai* sebagai strategi utama. Keputusan ini didasarkan pada kompleksitas pemecahan masalah (fase *decision-making*) yang jauh lebih adaptif.

Jika bot lain hanya menggunakan aturan *greedy* linear berbasis jarak statis yang sangat rentan terbaca oleh musuh, *RyoikiTenkai* memecah optimasi lokal per *turn* menjadi dua poros utama: Anti-Gravity untuk meminimalkan risiko ancaman posisi, dan N-Gram State Prediction untuk memaksimalkan efisiensi akurasi tembakan.

BAB 4

Implementasi dan Pengujian

4.1 Implementasi Strategi

4.1.1 Strategi Utama Kamikaze

Ketika *radar* berhasil mendeteksi musuh melalui *event* OnScannedBot, bot Kamikaze langsung menerapkan strategi *greedy* agresif. Bot akan menghitung sudut absolut musuh, memutar seluruh badan (*body*) dan senjata (*gun*) ke arah target, melepaskan tembakan dengan daya maksimal, lalu langsung menerjang maju ke posisi musuh.

Bot akan memprioritaskan untuk bertabrakan dengan target yang ada. Ketika bertabrakan dengan bot lawan (OnHitBot), Kamikaze tidak mundur melainkan memanfaatkan momentum jarak dekat tersebut secara *greedy* untuk terus menekan dan menembak dengan daya destruktif tertinggi.

4.1.2 Strategi Utama RotasiBumi

Bot RotasiBumi dirancang dengan pendekatan *greedy rule-based* yang berfokus pada fleksibilitas pergerakan berdasarkan jarak musuh (menyerang, mengorbit, dan menjauh), serta optimalisasi akurasi penembakan. Berbeda dengan bot Kamikaze, RotasiBumi mengaktifkan fitur AdjustGunForBodyTurn = true agar arah pergerakan senjata (*gun*) tidak terpengaruh oleh perputaran badan tank (*body*), sehingga proses pembidikan target lokal menjadi lebih independen dan presisi.

Ketika target berhasil terpindai oleh radar, fungsi OnScannedBot akan langsung memproses kalkulasi sudut lokal dan memisahkan aksi penembakan serta pergerakan secara *greedy* sebagai berikut:

FUNCTION on_scanned_bot(enemy):

```
// 1. EVALUASI JALUR BIDIKAN DAN JARAK LOKAL
bearing ← gun_bearing_to(enemy.x, enemy.y)
distance ← distance_to(enemy.x, enemy.y)

set_turn_gun_left(bearing)

// 2. EKSEKUSI PENEMBAKAN SECARA GREEDY (FUNGSI KELAYAKAN)
// Tembakan dilepaskan hanya jika deviasi sudut < 5 derajat dan senjata tidak panas
IF ABS(bearing) < 5 AND gun_heat = 0 THEN
    // Power ditentukan secara greedy: maksimal 3, atau mengecil seiring jarak demi
    efisiensi
    fire_power ← MIN(3, 400 / distance)
    fire(fire_power)
END IF
```

```
// 3. SELEKSI PERGERAKAN BERDASARKAN ATURAN JARAK (RULE-BASED
MOVEMENTS)
IF distance > 300 THEN
    // Kondisi Jarak Jauh: Kejar target secara linear untuk memperpendek jarak
    serang
    angle ← bearing_to(enemy.x, enemy.y)
    set_turn_left(angle)
    set_forward(distance - 150)

    ELSE IF distance > 200 THEN
        // Kondisi Jarak Sedang: Lakukan pergerakan sedikit mengorbit (sudut 20 derajat)
        set_turn_left(20)
        set_forward(100)

    ELSE
        // Kondisi Jarak Dekat/Terlalu Intim: Berputar tajam 90 derajat untuk
        menjauh/mengorbit total
        set_turn_left(90)
        set_forward(200)
    END IF

    go()
END FUNCTION
```

Untuk mengamankan bot dari kesalahan kalkulasi lokal yang stagnan , fungsi *event* responsif diimplementasikan guna menjaga mobilitas tank tetap tinggi di arena pertempuran.

```
FUNCTION on_hit_bot(enemy):
    // Mengambil keuntungan jarak 0 secara greedy dengan menembak power maksimal
    set_forward(100)
    fire(3)
    go()
END FUNCTION

FUNCTION on_hit_wall():
    // Mekanisme lepas dari dinding: Mundur lalu berputar balik kiri secara instan
    back(60)
    turn_left(180)
END FUNCTION
```

4.1.3 Strategi Utama Sweep

Bot Sweep dibangun menggunakan pendekatan *greedy* yang berfokus pada efisiensi pemindaian (*radar tracking*) dan penentuan daya tembak dinamis berdasarkan jarak target lokal. Menggunakan konfigurasi independen `AdjustGunForBodyTurn = true` dan `AdjustRadarForGunTurn = true`, pergerakan *radar* dan senjata (*gun*) terisolasi sepenuhnya dari rotasi *body* tank. Strategi awal yang unik dari bot ini adalah langsung mencari dinding arena terdekat pada saat *inisialisasi* awal pertempuran guna meminimalisasi sudut serang dari arah belakang.

Bot ini akan mencari tembok sebagai langkah pertama. Berikut adalah teknis dilakukannya hal tersebut:

FUNCTION `calc_angle_to_nearest_wall()`:

```
// 1. HITUNG JARAK KE EMPAT SISI DINDING ARENA
dist_left ← current_x
dist_right ← arena_width - current_x
dist_bottom ← current_y
dist_top ← arena_height - current_y

// 2. SELEKSI GREEDY UNTUK Mencari Jarak Minimum (Kandidat Terbaik)
min_dist ← MIN(MIN(dist_left, dist_right), MIN(dist_bottom, dist_top))

// 3. PEMETAAN SUDUT HEADING TARGET BERDASARKAN HASIL MINIMUM
IF min_dist = dist_left THEN
    target_heading ← 270
ELSE IF min_dist = dist_right THEN
    target_heading ← 90
ELSE IF min_dist = dist_bottom THEN
    target_heading ← 180
ELSE
    target_heading ← 0
END IF

// 4. NORMALISASI SUDUT RELATIF BEARING (-180 SAMPAI 180 DERAJAT)
bearing ← target_heading - current_direction

IF bearing > 180 THEN
    bearing ← bearing - 360
ELSE IF bearing < -180 THEN
    bearing ← bearing + 360
END IF

RETURN bearing
```

END FUNCTION

Ketika target musuh tertangkap oleh radar, Sweepy menerapkan fungsi kelayakan tembakan terukur berbasis batas jarak (*threshold-based greedy bullet power*):

FUNCTION on_scanned_bot(enemy):

```
// Menyimpan data bot terpindai ke dalam state lokal
scanned_target ← enemy

dist ← distance_to(enemy.x, enemy.y)

// SELEKSI DAYA TEMBAK SECARA GREEDY BERDASARKAN AMBANG JARAK
IF dist < 150 THEN
    bullet_power ← 3
ELSE IF dist < 300 THEN
    bullet_power ← 2
ELSE
    bullet_power ← 1
END IF

// Mengunci posisi senjata dan radar langsung ke arah koordinat musuh
set_turn_gun_left(gun_bearing_to(enemy.x, enemy.y))
set_turn_radar_left(radar_bearing_to(enemy.x, enemy.y))

// FUNGSI KELAYAKAN: Tembak hanya jika hawa panas komponen senjata bernilai 0
IF gun_heat = 0 THEN
    fire(bullet_power)
END IF
```

END FUNCTION

Untuk menjaga konsistensi target dan mobilitas taktis agar tidak terjebak di ujung arena, diimplementasikan beberapa fungsi *event handling* sebagai berikut:

// Reset referensi target lokal jika bot yang sedang diincar tereliminasi

FUNCTION on_bot_death(victim):

```
IF scanned_target ≠ NULL AND victim.id = scanned_target.id THEN
    scanned_target ← NULL
```

```
END IF
```

END FUNCTION

// Reaksi Greedy saat terjadi tabrakan: Mengunci radar dan menembak power maksimal (3)

FUNCTION on_hit_bot(enemy):

```
set_turn_radar_left(radar_bearing_to(enemy.x, enemy.y))
```

```
IF gun_heat = 0 THEN
```

```
    fire(3)
```

```
END IF
END FUNCTION
```

```
// Mitigasi benturan dinding: Putar badan tank 90 derajat menjauh secara instan
FUNCTION on_hit_wall():
    turn_left(90)
END FUNCTION
```

4.1.4 Strategi Utama RyoikiTenkai

Bot RyoikiTenkai dirancang dengan tingkat kompleksitas *greedy* yang jauh lebih adaptif dan taktis dibandingkan bot-bot sebelumnya. Jika bot lain menggunakan aturan linear berbasis jarak statis, RyoikiTenkai menerapkan strategi pergerakan berbasis medan gaya buatan (*Anti-Gravity Movement*) dan penargetan prediktif menggunakan riwayat data lokal (*N-Gram State Prediction*).

Seluruh komponen utama bot dikonfigurasi secara independen menggunakan properti `AdjustGunForBodyTurn`, `AdjustRadarForGunTurn`, dan `AdjustRadarForBodyTurn` bernilai `true` agar radar, senjata, dan badan tank dapat mengeksekusi keputusan *greedy* optimalnya masing-masing pada setiap *turn* tanpa saling menghambat.

Melalui *event* `OnTick`, bot melakukan evaluasi *greedy* spasial di sekelilingnya. Bot memetakan beberapa titik kandidat (berjumlah 36 arah koordinat) dengan radius tertentu (`MIN_RADIUS` hingga `MAX_RADIUS`). Fungsi objektif dari algoritma ini adalah mencari titik koordinat dengan tingkat bahaya (nilai gravitasi) paling minimum yang dihasilkan dari akumulasi energi musuh, garis lintasan peluru virtual, dan kedekatan sudut arena (*corner*).

```
FUNCTION on_tick_event():

    // 1. UPDATE ANIMASI WARNA DAN PERGERAKAN PELURU VIRTUAL LOKAL
    update_virtual_bullets_positions()

    // Jika dalam kondisi duel jarak dekat khusus, aktifkan Stop-and-Go (SAG)
    IF hitsag > SAG_LIMIT THEN dont_sub_sag ← TRUE END IF
    IF NOT dont_sub_sag AND enemy_count = 1 AND target_distance > 250 THEN RETURN
END IF

    best_x ← current_x
    best_y ← current_y
    min_gravity ← INFINITY

    // 2. EVALUASI HIMPUNAN KANDIDAT KOORDINAT SECARA GREEDY
    FOR i FROM 0 TO POINT_COUNT - 1 DO
        theta ← (2 × PI / POINT_COUNT) × i
```

```
FOR u FROM 0 TO 1 DO
  // Kalkulasi radius kandidat spasial
   $r \leftarrow \text{SQRT}(u \times (\text{MAX\_RADIUS}^2 - \text{MIN\_RADIUS}^2) + \text{MIN\_RADIUS}^2)$ 
  cand_x  $\leftarrow$  current_x + r * COS(theta)
  cand_y  $\leftarrow$  current_y + r * SIN(theta)

  // Fungsi Kelayakan: Amankan bot dari risiko menabrak dinding luar
  IF cand_x < WALL_MARGIN OR cand_x > (arena_width - WALL_MARGIN) OR
    cand_y < WALL_MARGIN OR cand_y > (arena_height - WALL_MARGIN) THEN
    CONTINUE
  END IF

  // Hitung bobot bahaya titik melalui Fungsi Gravitasi
  current_gravity  $\leftarrow$  calc_gravity_weight(cand_x, cand_y)

  // Seleksi lokal: Cari titik dengan nilai ancaman terkecil
  IF current_gravity < min_gravity THEN
    min_gravity  $\leftarrow$  current_gravity
    best_x  $\leftarrow$  cand_x
    best_y  $\leftarrow$  cand_y
  END IF
END FOR
END FOR

// 3. EKSEKUSI SOLUSI GERAKAN OPTIMUM LOKAL
IF min_gravity < calc_gravity_weight(dest_x, dest_y) * 0.9 THEN
  dest_x  $\leftarrow$  best_x
  dest_y  $\leftarrow$  best_y
END IF

// Eksekusi perputaran badan tank menuju titik teraman
turn_angle  $\leftarrow$  bearing_to(dest_x, dest_y) * PI / 180
set_turn_left(180 / PI * TAN(turn_angle))
set_forward(distance_to(dest_x, dest_y) * COS(turn_angle))
```

END FUNCTION

Ketika musuh terdeteksi, RyoikiTenkai tidak langsung menembak koordinat *statik* musuh. Bot mencatat riwayat perubahan orientasi (*angular velocity*), kecepatan (*speed*), dan percepatan (*acceleration*) target ke dalam pohon transisi data (*N-Gram Segment Tree*). Bot secara *greedy* memprediksi koordinat masa depan musuh berdasarkan rantai riwayat perilaku yang paling sering muncul (*most frequent state sequence*), lalu mengarahkan senjata tepat ke titik potong tersebut.

FUNCTION on_scanned_bot(enemy):

```
// 1. UPDATE KUNCIAN RADAR SECARA GREEDY
enemy_distance ← distance_to(enemy.x, enemy.y)
IF enemy_distance < target_distance THEN
    target_id ← enemy.id
END IF
target_distance ← enemy_distance

radar_angle ← INFINITY × normalize_angle(radar_bearing_to(enemy.x, enemy.y))
set_turn_radar_left(radar_angle)

// 2. KONTROL PENEMBAKAN BERDASARKAN AMBING ENERGI (FUNGSI
KELAYAKAN)
fire_power ← (current_energy / enemy_distance) × GUN_FACTOR
IF gun_turn_remaining = 0 AND (current_energy > MIN_ENERGY OR enemy_distance <
50) THEN
    set_fire(fire_power)
END IF

// 3. PEREKAMAN HISTORI PERILAKU GERAKAN MUSUH (NGRAM STATE)
current_state ← calculate_movement_state(enemy.speed, enemy.direction)
save_to_ngram_history(enemy.id, current_state)

// 4. SIMULASI PREDIKSI JALUR KOORDINAT (FORWARD SIMULATION)
pred_x ← enemy.x
pred_y ← enemy.y
time_tick ← 0
bullet_speed ← calc_bullet_speed(fire_power)

// Lakukan looping prediksi hingga waktu tempuh peluru berpapasan dengan posisi
simulasi musuh
WHILE (time_tick × bullet_speed) < distance_to(pred_x, pred_y) AND time_tick < 100
DO
    // Ambil keputusan status berikutnya dari data historis paling sering muncul
    next_predicted_state ← get_most_frequent_ngram_state(enemy.id)

    update_simulated_direction_and_speed(next_predicted_state)
    pred_x ← pred_x + predicted_speed × COS(predicted_direction)
    pred_y ← pred_y + predicted_speed × SIN(predicted_direction)
    time_tick ← time_tick + 1
END WHILE

// Batasi koordinat hasil prediksi agar tetap berada di dalam arena (Fungsi
Kelayakan)
pred_x ← MAX(WALL_MARGIN, MIN(arena_width - WALL_MARGIN, pred_x))
```

```
pred_y ← MAX(WALL_MARGIN, MIN(arena_height - WALL_MARGIN, pred_y))
```

```
// 5. BIDIK DAN ARAHKAN SENJATA KEDEPAN TARGET (PLAY IT FORWARD)
```

```
bearing_from_gun ← gun_bearing_to(pred_x, pred_y)
```

```
set_turn_gun_left(bearing_from_gun)
```

```
END FUNCTION
```

Guna mendukung akurasi prediksi internal, bot menyimulasikan tembakan peluru pelacak buatan sendiri melalui *event* `OnBulletFired` untuk mengukur efektivitas metode bidikan secara waktu nyata.

```
// Setiap kali senjata asli menembak, buat dua buah peluru virtual pelacak (tipe linear & tipe prediksi)
```

```
FUNCTION on_bullet_fired(bullet):
```

```
    add_my_virtual_bullet(current_x, current_y, bullet.speed, gun_direction, target_id, TYPE_PREDICTION)
```

```
    add_my_virtual_bullet(current_x, current_y, bullet.speed, linear_direction, target_id, TYPE_LINEAR)
```

```
END FUNCTION
```

```
// Jika terkena peluru dalam mode 1v1, naikan counter ancaman untuk memicu pergerakan Stop-and-Go
```

```
FUNCTION on_hit_by_bullet():
```

```
    IF enemy_count = 1 THEN
```

```
        hitsag ← hitsag + 1
```

```
    END IF
```

```
END FUNCTION
```

```
// Reset kunci target jika musuh utama tereliminasi dari arena
```

```
FUNCTION on_bot_death(victim):
```

```
    set_enemy_alive_status(victim.id, FALSE)
```

```
    IF victim.id = target_id THEN
```

```
        target_distance ← INFINITY
```

```
    END IF
```

```
END FUNCTION
```

BAB 5

Kesimpulan dan Saran

5.1. Kesimpulan

Berdasarkan hasil perancangan, implementasi, dan analisis terhadap empat model bot *Tank Royale* berbasis algoritma *greedy* yang telah dikembangkan, dapat ditarik beberapa kesimpulan sebagai berikut:

1. Bot Kamikaze merepresentasikan penerapan strategi *Greedy Agresif* yang murni dan sederhana. Dengan memprioritaskan serangan jarak dekat secara konstan (*fire(3)*) dan langsung menerjang ke arah koordinat musuh (*OnScannedBot*), bot ini sangat efektif dalam memberikan tekanan psikologis (*ramming*) pada pertempuran jarak dekat, namun rentan tereliminasi lebih awal jika menghadapi musuh dengan mobilitas tinggi atau taktik defensif.
2. Bot RotasiBumi menerapkan pendekatan *Greedy Rule-Based* berbasis evaluasi jarak spasial. Bot ini berhasil menutupi kelemahan *Kamikaze* dengan membagi keputusan gerak menjadi tiga kondisi (mengejar, mengorbit, atau menjauh tajam). Penggunaan fungsi matematika membuktikan bahwa fungsi seleksi kelayakan peluru secara lokal mampu meningkatkan efisiensi konsumsi energi bot secara signifikan.
3. Bot Sweepy menunjukkan keunggulan pada taktik pemanfaatan lingkungan (*Greedy Wall-Alignment*). Dengan melakukan kalkulasi *heuristic* awal untuk mencari dan merapat ke arah dinding terdekat (*CalcAngleToNearestWall*), bot ini berhasil meminimalkan sudut serang dari arah belakang. Selain itu, sistem penguncian radar independen serta manajemen kekuatan peluru bertingkat (3, 2, dan 1) membuat *Sweepy* menjadi bot pertahanan yang sangat stabil.
4. Bot RyoikiTenkai terbukti menjadi bot dengan strategi paling canggih, adaptif, dan seimbang (*Ultimate High-Tier Strategy*). Meskipun memanfaatkan arsitektur kompleks seperti medan gaya buatan (*Anti-Gravity Movement*) untuk navigasi teraman dan pohon transisi data (*N-Gram State Prediction*) untuk menembak titik masa depan target (*Play It Forward*), bot ini tetap bekerja di bawah koridor *greedy* karena seluruh pengambilan keputusan terbaik hanya dievaluasi secara lokal per *turn* berjalan.
5. Secara keseluruhan, pengujian membuktikan bahwa algoritma *greedy* yang dikombinasikan dengan fungsi seleksi berbasis data historis (*N-Gram*) dan perhitungan kalkulasi spasial (*Anti-Gravity*) jauh lebih unggul dibandingkan aturan *greedy* linear statis dalam memenangkan pertempuran di arena Robocode.

5.2. Saran

Untuk pengembangan penelitian dan optimalisasi performa bot pada masa mendatang, beberapa poin saran yang dapat diimplementasikan adalah:

1. Integrasi Logika Prediksi pada Bot Linear:

Bot dengan mobilitas tinggi seperti *RotasiBumi* atau *Sweepy* dapat ditingkatkan efektivitas tembakannya jika fungsi penargetannya diadopsi dari metode prediksi *N-Gram* milik *RyoikiTenkai*, sehingga peluru mereka tidak mudah meleset saat menghadapi musuh yang bergerak melingkar.

2. Penerapan Algoritma Dinamis (Dynamic Programming/Look-Ahead):

Kelemahan utama dari seluruh bot berbasis *greedy* lokal ini adalah ketidakmampuannya mengantisipasi jebakan jangka panjang. Pengembangan selanjutnya dapat menerapkan algoritma *Minimax* atau *Q-Learning* sederhana agar tank dapat memproyeksikan beberapa *turn* ke depan (*global planning*).

3. Penyempurnaan Fungsi Bobot Anti-Gravity:

Pada bot *RyoikiTenkai*, perhitungan kalkulasi medan gravitasi berbahaya dapat diperluas dengan menambahkan variabel sisa energi musuh dan arah hadap moncong senjata lawan, sehingga bot dapat membedakan mana musuh yang sedang lengah dan mana musuh yang siap menembak.

4. Pengambilan Data Kuantitatif yang Lebih Komprehensif:

Pengujian berikutnya disarankan untuk dilengkapi dengan visualisasi grafik statistik yang mencakup metrik *win-rate* (rasio kemenangan), *bullet accuracy* (akurasi tembakan), sisa energi rata-rata, serta total waktu bertahan hidup (*survival time*) melalui simulasi pertempuran minimal 100 ronde untuk mendapatkan kesimpulan data yang lebih valid secara empiris.

Lampiran

Link Repository Git: https://github.com/ChiruNukai/TubesStima2026-Kelompok_MBG_MicayamBiniGue

Link Drive ZIP:

Daftar Pustaka

Chen, X. (2022). A Comparison of Greedy Algorithm and Dynamic Programming Algorithm. *SHS Web of Conferences*, 144, 03009.
<https://doi.org/10.1051/shsconf/202214403009>